UNIVERSIDAD PERUANA UNIÓN

FACULTAD DE INGENIERÍA Y ARQUITECTURA

E.P. DE INGENIERÍA DE SISTEMAS



INFORME DE SEGURIDAD PHARMABACKEND

CURSO:
Lenguaje de Programación II

DOCENTE:
Ing. Benjamín David Reyna Barreto

ESTUDIANTE:
Mishelle Esmeralda Osorio Huashuayo

LIMA, OCTUBRE 2026

INFORME DE SEGURIDAD DE PHARMABACKEND

Actividad autónoma · Sesión 10 · API local evaluada con la matriz y las evidencias entregadas.

C1. Resumen de resultados

Se registraron 28 casos: 27 ejecutados y uno no ejecutado. De los ejecutados, 23 pasan, dos son parciales y dos figuran como fallidos según la matriz. Los parciales no se cuentan como aprobados porque falta verificar criterios solicitados.

Grupo	Ejecutados	Pasan	Parciales	Fallan	No ejec.
Autenticación	5	5	0	0	0
Tokens	6	5	0	1	0
Permisos	5	5	0	0	0
Usuarios	2	1	1	0	1
CORS	1	1	0	0	0
Contraseña	4	3	1	0	0
Roles	2	2	0	0	0
Estado / roles	1	1	0	0	0
Estado	1	0	0	1	0
Total	27	23	2	2	1

C2. Fichas de hallazgos

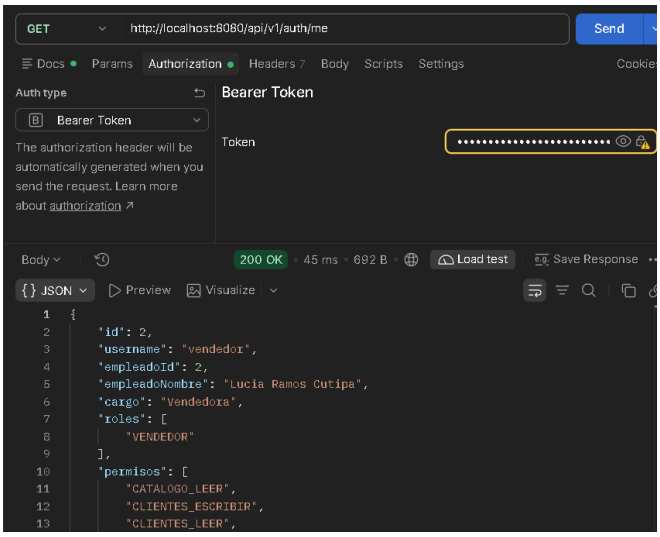
T-05 · Verificación de claims del JWT | Severidad: Baja (evidencia incompleta)

Descripción: La prueba muestra GET /auth/me con 200, pero no verifica los claims requeridos ni demuestra que el payload carezca de datos sensibles. No se confirma una vulnerabilidad en el token.

Evidencia: Evidencias_Actividad_Autonoma_S10.docx, p. 8; matriz T-05.

Causa probable: La captura corresponde a AuthController.me(); no incluye la inspección del JWT generado por JwtService.generarToken(UserDetails).

Corrección propuesta: Decodificar localmente el payload y registrar sub, authorities, iat y exp, ocultando el token completo. Comparar con JwtService.generarToken y corregir el generador solo si los claims observados no coinciden con el diseño.



Captura T-05: GET /auth/me responde 200; no incluye la decodificación de los claims.

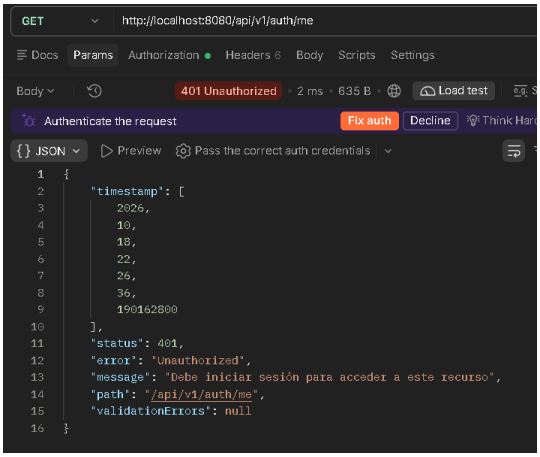
B-08 · Mensaje con token de cuenta desactivada | Severidad: Baja (resultado no concluyente)

Descripción: El token anterior recibió 401, por lo que la petición fue denegada; la respuesta mostró el mensaje genérico en vez de “El usuario está deshabilitado”. La evidencia no confirma por sí sola el motivo específico del rechazo.

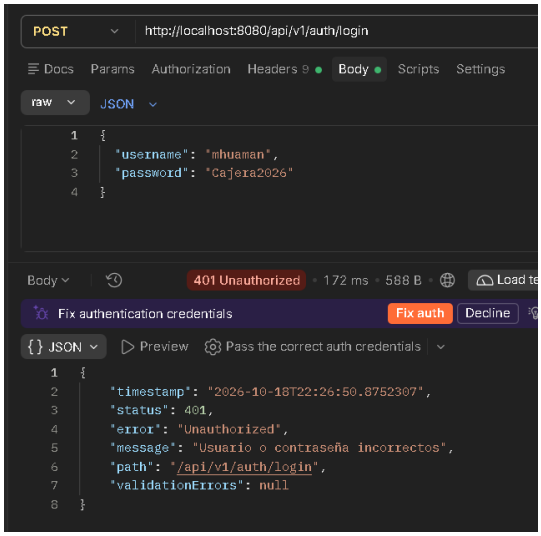
Evidencia: Evidencias_Actividad_Autonoma_S10.docx, pp. 25–27; matriz B-08.

Causa probable: JwtAuthenticationFilter.doFilterInternal asigna jwt.error si UserDetails.isEnabled() es falso; RestAuthenticationEntryPoint.commence usa ese motivo o, si falta, responde el texto genérico. Puede que la rama no se activara o que el token/header de prueba fuera incorrecto; la causa no está confirmada.

Corrección propuesta: Repetir con un token válido emitido antes de desactivar la cuenta y enviarlo como Bearer. Verificar que se recorra la rama de usuario deshabilitado y que el entry point lea jwt.error; si persiste el mensaje genérico, ajustar ese flujo y agregar una prueba de integración.



Token previo: 401 con mensaje genérico.



Login con cuenta desactivada: 401.

C3. Preguntas de análisis

1. ¿Por qué se usa el mismo mensaje para usuario inexistente, contraseña incorrecta y cuenta deshabilitada? ¿Qué revelaría uno distinto?

Una respuesta uniforme evita la enumeración de cuentas: si el sistema distingue entre usuario inexistente, clave incorrecta y cuenta deshabilitada, un atacante puede descubrir qué nombres son válidos y el estado de cada cuenta para dirigir ataques posteriores. En esta evidencia, AuthServiceImpl.login delega la autenticación y las capturas A-02/A-03 muestran 401 con “Usuario o contraseña incorrectos”; mantener el texto común reduce filtración de información, aunque no reemplaza límites de intentos ni monitoreo.

2. Tras cambiar la contraseña, ¿por qué sigue válido el token anterior y cómo se puede invalidar?

La API configura sesiones STATELESS en SecurityConfig: el servidor no conserva una sesión que pueda cerrar al cambiar la contraseña. El JWT conserva una firma válida y su vencimiento hasta expirar; AuthServiceImpl.cambiarPassword actualiza la clave, pero el filtro no comprueba una versión de token. Una opción es agregar tokenVersion al usuario, incluirla en el JWT y compararla con el valor vigente en cada petición; incrementar la versión al cambiar la contraseña invalida los tokens anteriores. Otra opción es una lista de revocación por jti, con el costo de consultar estado en cada solicitud.

3. Si el secreto JWT se publica en GitHub, ¿qué podría hacer un atacante y qué acciones inmediatas corresponden?

Con el secreto puede firmar JWT falsos y suplantar sujetos con cuentas existentes; en este proyecto el filtro vuelve a cargar al usuario desde la base de datos, así que sus permisos efectivos provienen del usuario vigente. De inmediato se debe rotar el secreto, reiniciar el servicio para que deje de aceptar tokens firmados con la clave anterior, forzar nuevos inicios de sesión, revisar accesos y registros, y retirar el secreto del código y del historial. Aunque se borre del repositorio, debe tratarse como comprometido.

4. ¿Qué ventaja y desventaja tiene @PreAuthorize en el controlador frente a una regla por URL en SecurityConfig?

En ReporteController, @PreAuthorize("hasAuthority('REPORTES_VER')") expresa la política junto al controlador y protege métodos con una autoridad específica; evita depender únicamente del mapeo de una ruta en la configuración web. La desventaja es que las reglas quedan distribuidas entre controladores y requieren habilitar seguridad de métodos; son menos centralizadas y deben considerar proxies y llamadas internas. SecurityConfig centraliza reglas por URL y método, pero pueden ser más gruesas y una ruta nueva puede quedar mal cubierta si no se actualizan los matchers.

Base del informe: guía autónoma S10, matriz y evidencias recibidas; implementación inspeccionada en JwtService, JwtAuthenticationFilter, RestAuthenticationEntryPoint, SecurityConfig, AuthServiceImpl y ReporteController.